

Experiment No.	9
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning
Student Name	Chitraju Vishnu Vineeth
Register Number	3122247001012
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	20/09/2026

1 Objective

The aim of this experiment is to compare two classifiers on the English Handwritten Characters dataset. Model A is a single-layer Perceptron Learning Algorithm (PLA) written from scratch with a step activation. Model B is a multilayer perceptron (MLP) with hidden layers and non-linear activations. The MLP hyperparameters (activation function, cost function, optimizer, learning rate, batch size and number of hidden layers) are chosen by a systematic search, and the two models are compared using accuracy, precision, recall, F1-score, confusion matrices, ROC curves and training-error curves.

2 Problem Statement

Input: a grayscale image of one handwritten character, resized to 20×20 pixels and flattened into a vector of 400 values between 0 and 1.

Output: one of 62 classes (digits 0–9, upper-case A–Z and lower-case a–z).

Objective: learn a mapping from the 400 pixel values to the class label, find out how far a linear single-layer model can go, and measure how much a tuned MLP improves on it.

3 Dataset Description

Dataset Name	English Handwritten Characters
Dataset Source	Kaggle (dhruvildave/english-handwritten-characters-dataset), built from the Chars74K handwritten set [?, 3]
Number of Samples	3,410 images
Number of Features	400 (20×20 pixels after resizing; the original images are 1200×900 RGB)
Number of Classes	62 (55 images per class)
Missing Values	None
Train-Test Split	80% train / 20% test, stratified: 2,728 train and 682 test images. For tuning, the training part is split again 80/20 into 2,182 train and 546 validation images.

4 Exploratory Data Analysis

4.1 Dataset Overview

The dataset has a CSV file with two columns (image path and label) and 3,410 PNG images in the folder `Img`. All images are 1200×900 RGB, with a black pen stroke on a white background. Figure 1 shows ten images after preprocessing.

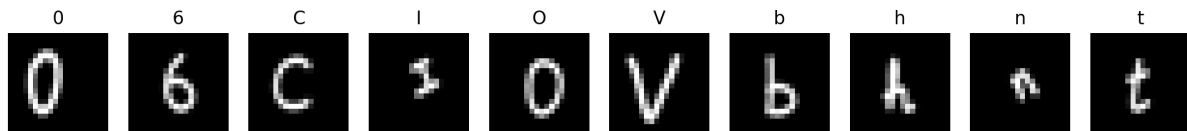


Figure 1: Ten sample images after grayscale conversion, resizing to 20×20 and inversion.

Inference: The same kind of character is drawn at very different sizes and stroke widths; for example the “l” and the “n” above are small, while the “0” and “O” fill most of the box. The pen stroke covers only the middle part of the 20×20 grid, so each character is described by a small number of active pixels. Shapes such as “0” and “O” look almost the same, which suggests that some classes will be hard to separate.

4.2 Statistical Summary

Table 2 summarises all 3410×400 pixel values after scaling to $[0, 1]$.

Statistic	Value
Count	1,364,000
Mean	0.060
Standard deviation	0.199
Minimum	0.000
25% / 50% / 75%	0.000 / 0.000 / 0.000
Maximum	1.000
Share of stroke pixels (> 0.5)	5.9%

Table 2: Statistical summary of the pixel values.

Even the 75th percentile is zero, so at least three quarters of all pixel values are background.

4.3 Missing Value Analysis

Both the label file and the pixel matrix were checked. There are no missing labels or image paths and no missing pixel values (0 in both cases), so no imputation was needed.

4.4 Class Distribution

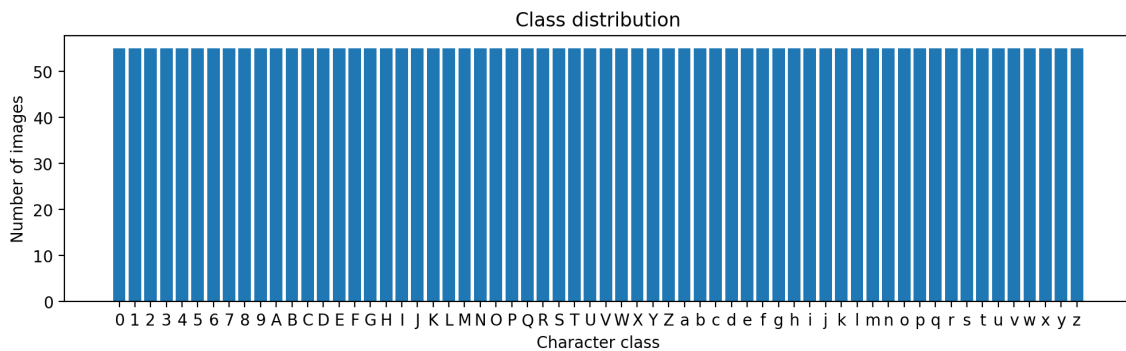


Figure 2: Number of images in each of the 62 classes.

Inference: Every class has exactly 55 images, so the dataset is perfectly balanced. Accuracy is therefore a fair metric here and no resampling is needed. Random guessing would give only about 1.6% accuracy ($1/62$). With a stratified split, each class has 11 test images, so the accuracy of a single class can only change in steps of about 9%.

4.5 Correlation Matrix

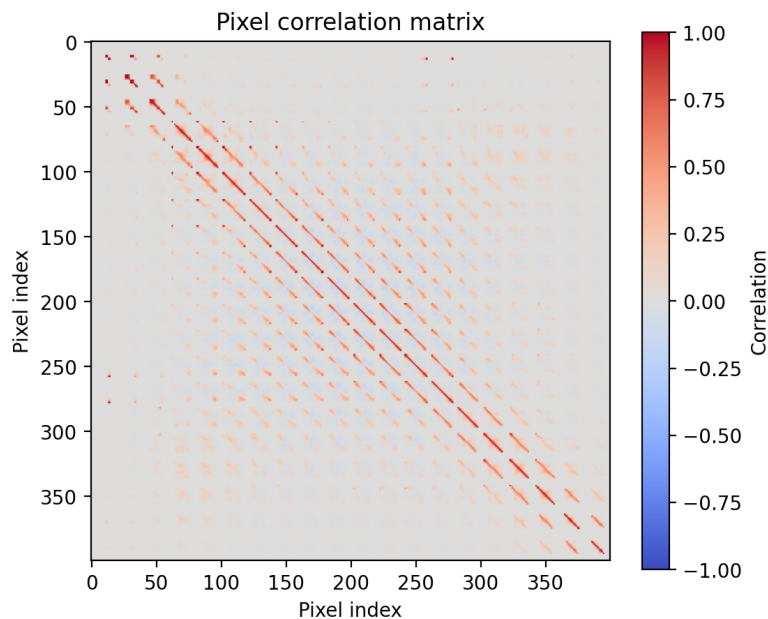


Figure 3: Correlation matrix of the 400 pixel features.

Inference: Most pixel pairs have a correlation close to zero (grey area). The strong values lie on a thin diagonal band, which means a pixel is mostly related to its immediate neighbours along the stroke. The same band repeats every 20 indices, which is the neighbouring row of the image. Strong negative correlations are almost absent. No small group of pixels is enough to identify a character, so the classifier has to combine many pixels.

4.6 Feature Distribution

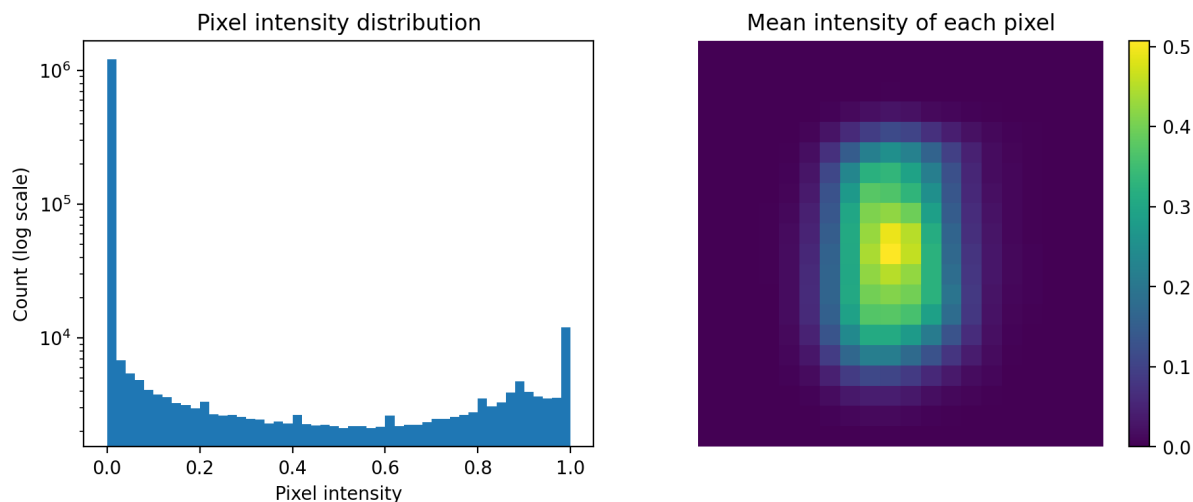


Figure 4: Left: histogram of all pixel values (log scale). Right: mean intensity of each pixel over the whole dataset.

Inference: The histogram has a very large peak at 0 (background) and a smaller peak at 1 (centre of the pen stroke); the values in between come from the smoothing done by resizing. The mean image is bright only in the central region, and the outer border is almost always zero. Border pixels therefore carry almost no information, and they also appear as empty rows and columns in the correlation matrix.

5 Data Preprocessing

- **Missing value handling:** not required, as no value is missing.
- **Encoding:** the 62 character labels were converted to integers 0–61. For the PLA targets and for the ROC and precision-recall curves, the labels were also converted to one-hot vectors.
- **Feature scaling:** each image is converted to grayscale, inverted (stroke = 1, background = 0) and divided by 255, so all values lie in $[0, 1]$.
- **Train-test split:** stratified 80/20 split (2,728 / 682 images). The training part was split again 80/20 (2,182 / 546) to get a validation set for tuning. The test set was used only once, for the final comparison.
- **Feature engineering:** images were resized to 20×20 and flattened into 400 features. No other features were created.

6 Machine Learning Algorithm

6.1 Model A: Perceptron Learning Algorithm (PLA)

Working principle. The perceptron [1] computes a weighted sum of the inputs and passes it through a step function. Because there are 62 classes, one perceptron is used per

class (one-vs-rest), giving 62 output neurons. During training, the samples are visited one by one in random order and the weights change only when a neuron gives a wrong output. At prediction time, the class whose neuron has the largest net input is selected.

Mathematical formulation. For class k and input $\mathbf{x}$,

$$z_k = \mathbf{w}_k^\top \mathbf{x} + b_k, \quad \hat{y}_k = \begin{cases} 1 & z_k \geq 0 \\ 0 & z_k < 0 \end{cases} \quad (1)$$

$$\mathbf{w}_k \leftarrow \mathbf{w}_k + \eta (y_k - \hat{y}_k) \mathbf{x}, \quad b_k \leftarrow b_k + \eta (y_k - \hat{y}_k), \quad \hat{c} = \arg \max_k z_k \quad (2)$$

where $y_k = 1$ for the true class and 0 otherwise, and η is the learning rate.

Advantages. It is simple, very fast to train, needs no derivatives, and is easy to explain.

Limitations. It can only draw linear boundaries, so it converges only when the classes are linearly separable. Its output is a hard 0/1 value, so it gives no class probabilities.

6.2 Model B: Multilayer Perceptron (MLP)

Working principle. An MLP has an input layer, one or more hidden layers with a non-linear activation, and an output layer. The hidden layers build new features from the pixels, which lets the network learn non-linear decision boundaries. The weights are learned by backpropagation [2], which sends the error backwards through the layers and updates the weights by gradient descent [?].

Mathematical formulation. For layer l with weights $W^{(l)}$ and bias $\mathbf{b}^{(l)}$,

$$\mathbf{a}^{(l)} = \varphi(W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}), \quad p_k = \frac{e^{z_k}}{\sum_{j=1}^{62} e^{z_j}} \quad (3)$$

where φ is ReLU, tanh or sigmoid, and the output layer uses softmax. The cost function is the cross-entropy over N training samples,

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^{62} y_{ik} \log p_{ik}. \quad (4)$$

With softmax and cross-entropy the output error is $\boldsymbol{\delta}^{(L)} = \mathbf{p} - \mathbf{y}$, and it is propagated backwards as $\boldsymbol{\delta}^{(l)} = (W^{(l+1)\top} \boldsymbol{\delta}^{(l+1)}) \odot \varphi'(\mathbf{z}^{(l)})$. The gradient of the weights is $\partial L / \partial W^{(l)} = \boldsymbol{\delta}^{(l)} \mathbf{a}^{(l-1)\top}$. SGD updates a weight as $\theta \leftarrow \theta - \eta \nabla_{\theta} L$, whereas Adam [?] keeps running averages of the gradient and its square and scales the step for every weight.

Advantages. It learns non-linear patterns, gives class probabilities, and works with different activations, optimizers and depths.

Limitations. It has many hyperparameters to tune, needs more computation than PLA, and overfits easily when the training set is small.

7 Experimental Setup

Python Version	3.12.3
Scikit-Learn Version	1.8.0 [?]
IDE	Jupyter Notebook
Operating System	Ubuntu 24.04 LTS (Linux)
Hardware	Intel Xeon @ 2.10 GHz, 1 vCPU, 3 GB RAM

The complete notebook (loading the images, tuning 108 MLP configurations, final training and all plots) runs in about five minutes on this hardware.

8 Hyperparameter Selection

8.1 PLA

Parameter	Value
Activation	Step (threshold at 0)
Learning rate η	0.1
Epochs	30
Initial weights and bias	0
Sample order	Shuffled in every epoch

With zero initial weights, η only rescales the weights and does not change the sign of z_k , so the value of η has no effect on the PLA result.

8.2 MLP: Search Space

Every combination in Table 3 ($3 \times 2 \times 3 \times 2 \times 3 = 108$ runs) was trained for 50 epochs on the train part and scored on the validation part. The last column gives the mean validation accuracy over all runs that use that value.

Hyperparameter	Values tried (mean validation accuracy)	Chosen
Activation	ReLU (0.321), Tanh (0.327), Sigmoid (0.192)	ReLU
Optimizer	SGD (0.259), Adam (0.301)	Adam
Learning rate	0.001 (0.246), 0.01 (0.372), 0.1 (0.222)	0.01
Batch size	32 (0.298), 128 (0.262)	128
Hidden layers	1: (128) (0.329), 2: (128, 64) (0.273), 3: (128, 64, 32) (0.238)	(128, 64)

Table 3: Search space, mean validation accuracy per value, and the chosen value.

SGD was run with scikit-learn's default momentum of 0.9 (Nesterov), and Adam with its default $\beta_1 = 0.9$ and $\beta_2 = 0.999$. The impact of a hyperparameter was measured as the

difference between the best and worst mean accuracy of its values: learning rate 0.151, activation 0.135, hidden layers 0.092, optimizer 0.043 and batch size 0.036.

8.3 MLP: Best Configurations

Rank	Activation	Optimizer	LR	Batch	Hidden layers	Val. acc.
1	ReLU	Adam	0.01	128	(128, 64)	0.518
2	ReLU	SGD	0.1	32	(128)	0.509
3	Sigmoid	Adam	0.01	128	(128, 64)	0.505
4	ReLU	SGD	0.01	32	(128, 64, 32)	0.502
5	ReLU	SGD	0.1	128	(128, 64)	0.498

Table 4: Five best configurations on the validation set.

8.4 MLP: Final Choice and Justification

Parameter	Value
Hidden layers	2 layers, 128 and 64 neurons
Activation	ReLU
Optimizer	Adam
Learning rate	0.01
Batch size	128
Cost function	Cross-entropy
Epochs	50
L2 penalty (α)	0.0001 (scikit-learn default)

- **Activation:** ReLU gave the best individual configurations. Tanh has a similar average, but Sigmoid is clearly worse (0.192), probably because its gradients become very small.
- **Optimizer:** Adam has the best single result and a higher average than SGD, since it adapts its step size and is less sensitive to the learning rate.
- **Learning rate:** 0.01 has the best average (0.372). At 0.001 training is too slow for 50 epochs, and at 0.1 the average falls to 0.222 because only some SGD runs still do well.
- **Batch size:** the effect is small (0.036). 128 was kept because it belongs to the best run and makes each epoch faster.
- **Hidden layers:** the best 1-, 2- and 3-layer networks score 0.509, 0.518 and 0.502, so depth hardly matters. Two layers gave the top score.
- **Cost function:** training the best configuration with mean squared error (an `MLPRegressor` on one-hot targets) gave only 0.075 validation accuracy, against 0.518 with cross-entropy. With 62 mostly-zero targets, the MSE gradients are very small and the network learns slowly.

9 Results

The final PLA and MLP were trained on the full training part (2,728 images) and evaluated on the 682 test images. An MLP with the untuned scikit-learn defaults is included to show the effect of tuning. Precision, recall, F1-score and ROC-AUC are macro averages over the 62 classes.

Metric	PLA	MLP (default)	MLP (tuned)
Accuracy	0.232	0.424	0.529
Precision	0.263	0.433	0.549
Recall	0.232	0.424	0.529
F1-score	0.209	0.418	0.531
ROC-AUC	0.825	0.912	0.954
Training accuracy	0.489	0.644	0.999

Table 5: Test-set results of the three models.

10 Result Visualization

The class distribution and correlation matrix are shown in Figures 2 and 3 (Section 4). The remaining graphs of the experiment follow.

10.1 Confusion Matrices

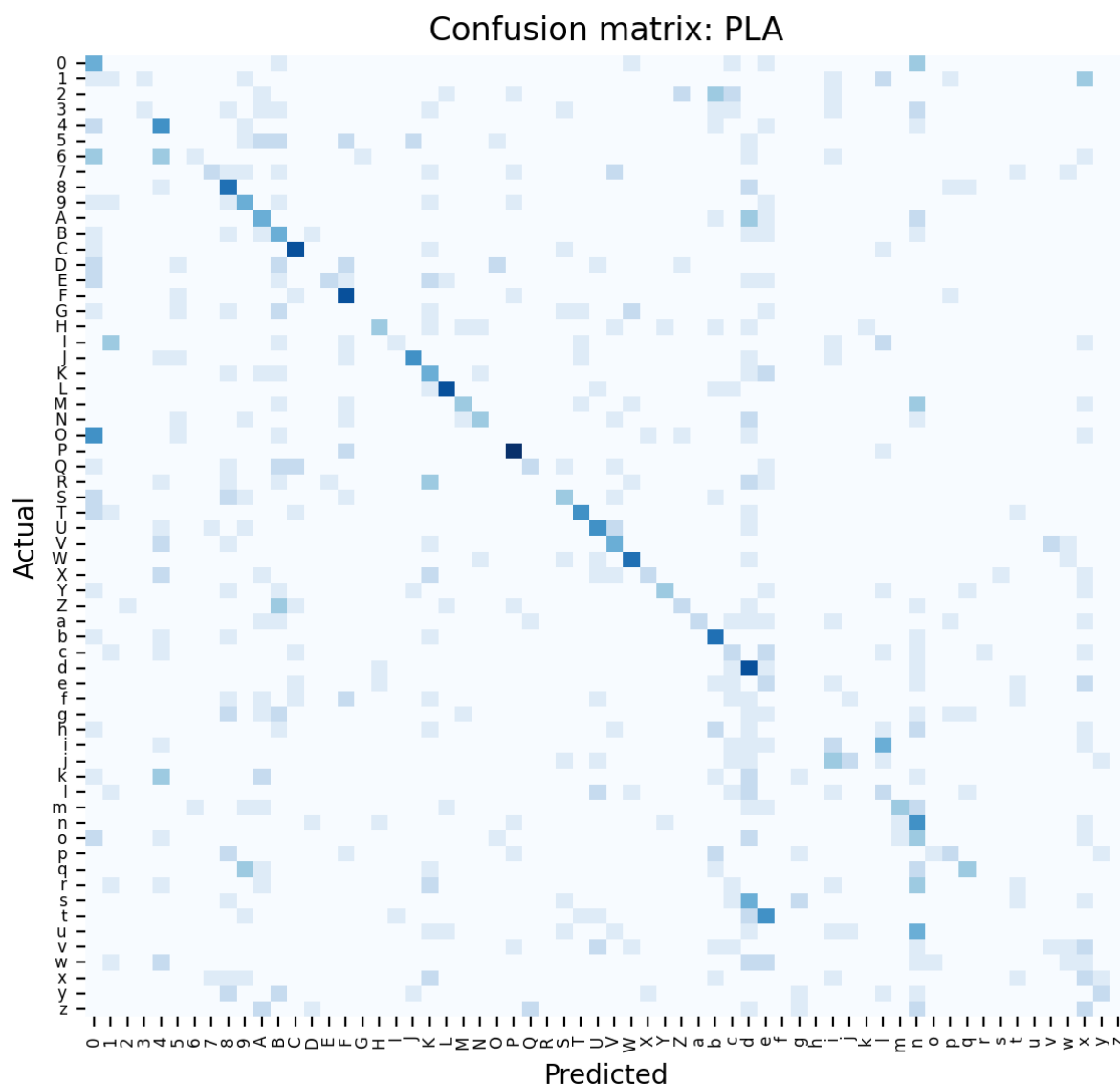


Figure 5: Confusion matrix of the PLA on the test set.

Inference: The diagonal of the PLA matrix is weak and the errors are spread over the whole matrix. A few classes attract far too many predictions: “d” is predicted 51 times, “n” 46 times, “0” 34 times and “B” 32 times, although every class has only 11 test images. Five classes are never predicted at all. The best class is “P” with 73% recall.

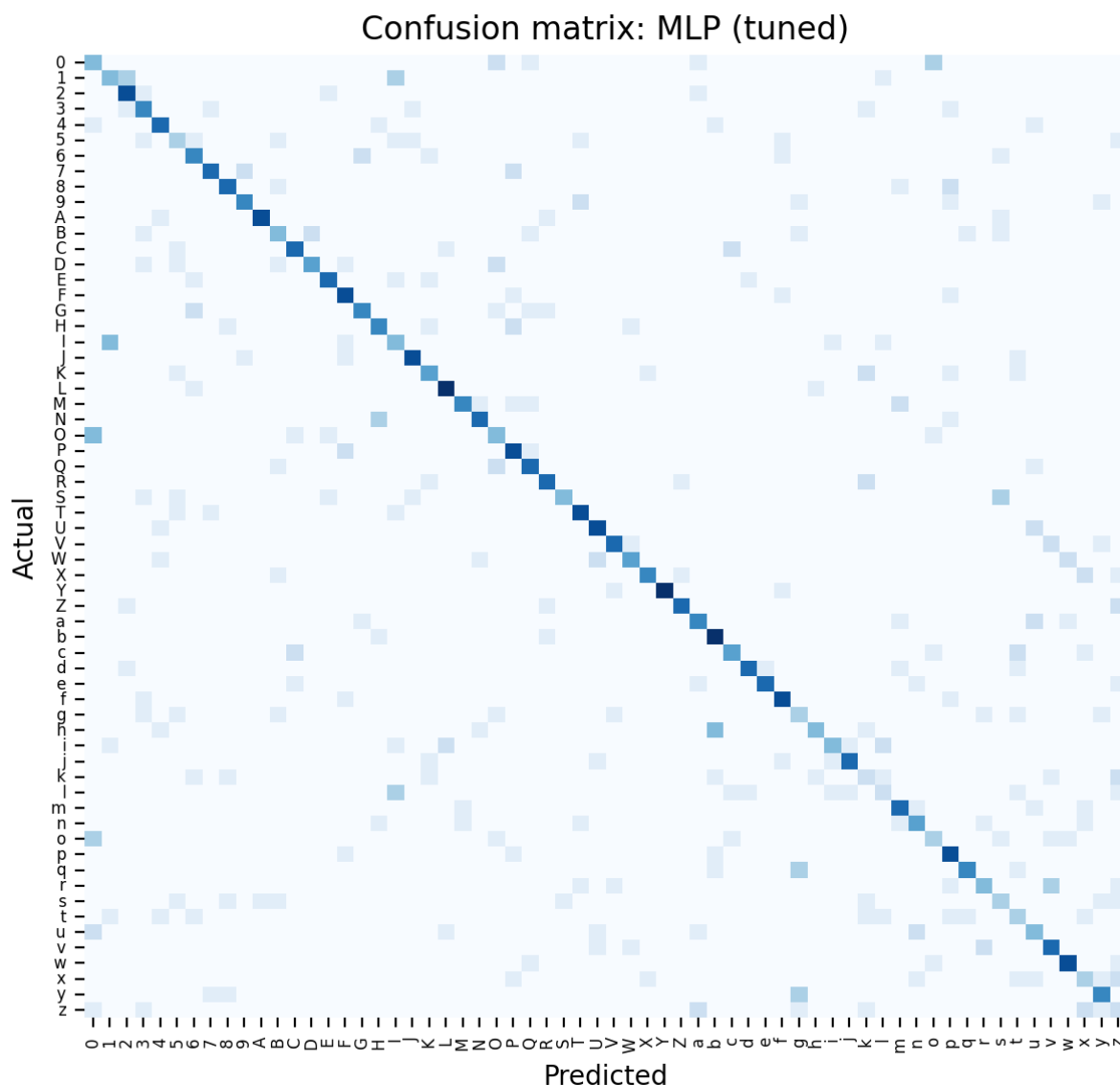


Figure 6: Confusion matrix of the tuned MLP on the test set.

Inference: The tuned MLP shows a clear diagonal, with 361 of 682 test images classified correctly. “L”, “Y” and “b” are recognised best (82%), and “k”, “z” and “l” worst (18%). The most common mistakes are between look-alike shapes: “I” and “l”, “O” and “0”, “h” and “b”, “q” and “g”. Of the 321 wrong predictions, 33 are the same letter in the wrong case.

10.2 ROC Curves

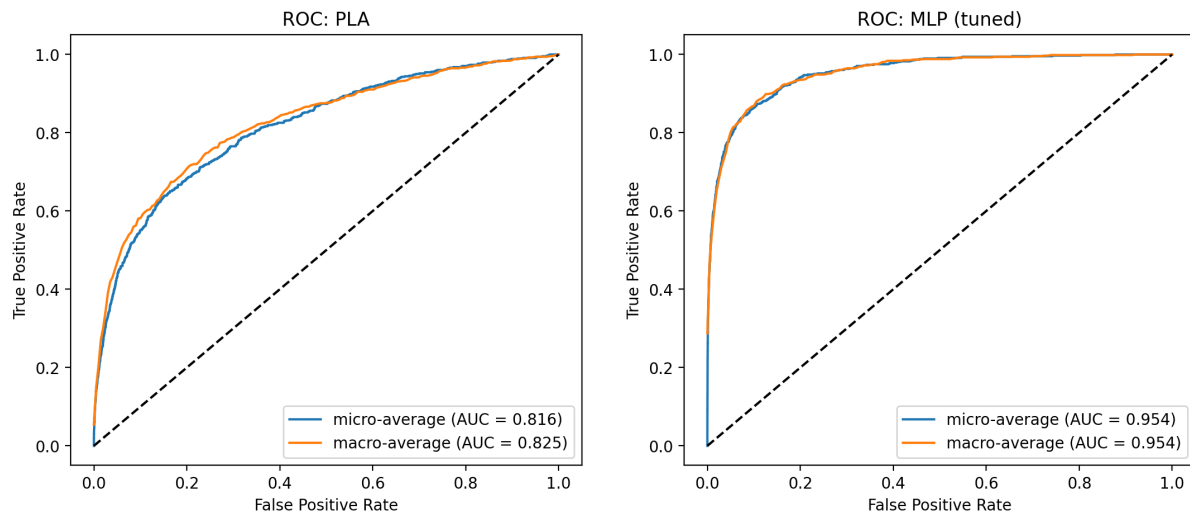


Figure 7: Micro- and macro-average ROC curves (one-vs-rest) of the PLA and the tuned MLP.

Inference: The macro-average AUC is 0.825 for the PLA and 0.954 for the MLP (micro-average: 0.816 and 0.954). The PLA curve lies above the diagonal, so its scores still rank the correct class better than chance, but it rises slowly. The MLP reaches a true positive rate of about 0.8 at a false positive rate of about 0.05. The AUC is high even though accuracy is only 53% because each one-vs-rest problem has 61 negative classes and the AUC measures ranking, not the top-1 decision.

10.3 Precision-Recall Curves

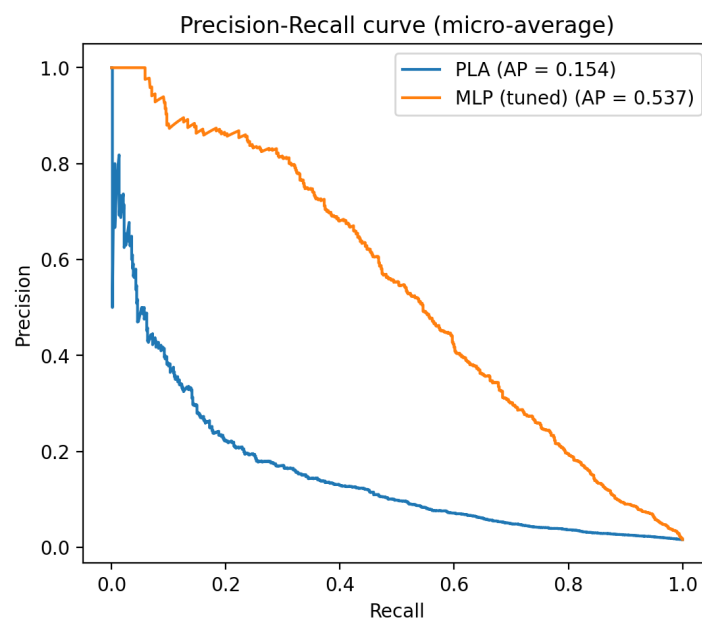


Figure 8: Micro-average precision-recall curves of the PLA and the tuned MLP.

Inference: The average precision is 0.537 for the MLP and only 0.154 for the PLA; a random scorer would get about 0.016. The MLP holds a precision near or above 0.8 until a recall of about 0.3 and then falls slowly. The PLA drops below 0.5 at a recall of less than 0.05. Since the PLA output is a raw net input and not a probability, its curve is also less smooth.

10.4 Learning Curves (Training Error vs Epochs)

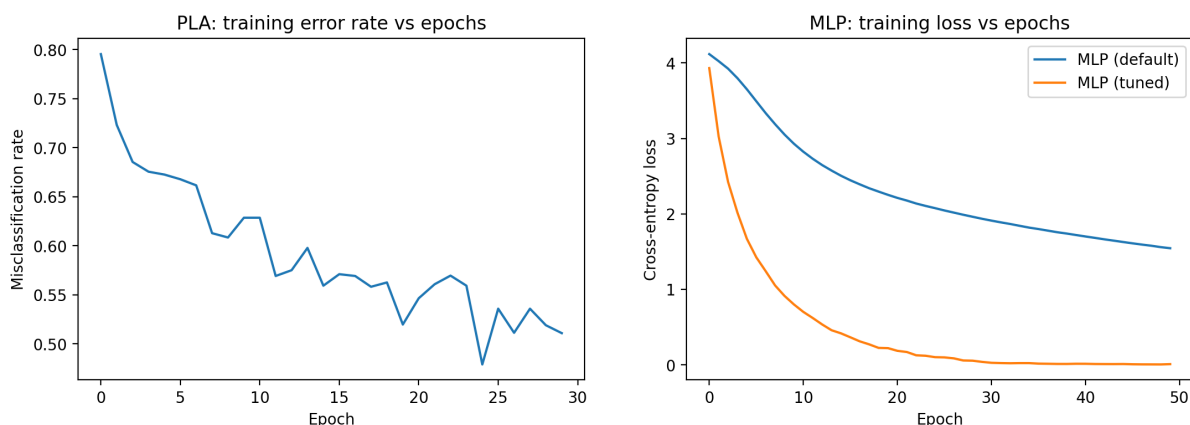


Figure 9: Left: PLA training error rate per epoch. Right: cross-entropy training loss of the default and tuned MLP.

Inference: The PLA error falls from 0.80 to about 0.51 in 30 epochs, with a lowest value of 0.48 near epoch 25, and it keeps jumping up and down instead of settling. This is what we expect when the classes are not linearly separable. The default MLP loss goes from 4.12 to 1.55 and is still falling after 50 epochs. The tuned MLP goes from 3.93 to 0.011 and is below 0.1 by about epoch 25, so tuning made the training much faster.

10.5 SGD vs Adam

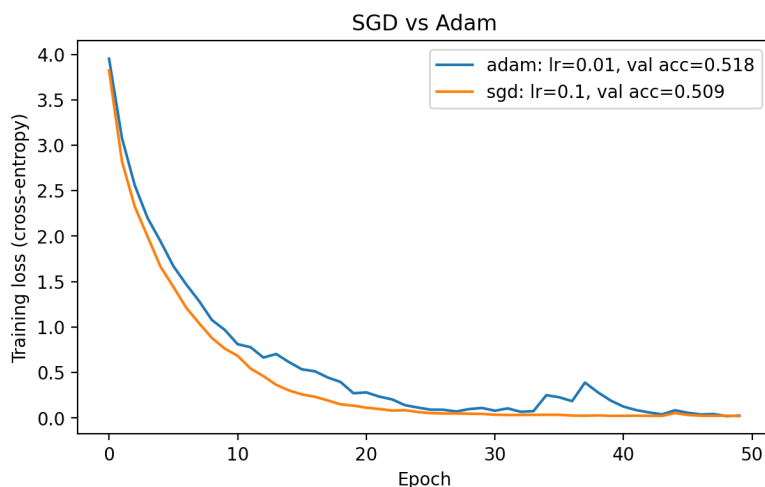


Figure 10: Training loss of the best SGD run and the best Adam run.

Inference: The best Adam run (learning rate 0.01, batch 128) reached 0.518 validation accuracy and the best SGD run (learning rate 0.1, batch 32) reached 0.509, a gap of about five images out of 546. SGD falls faster at the start, partly because batch size 32 gives four times more updates per epoch. The Adam curve shows a small bump around epochs 35–40 before it recovers. Both losses end close to zero.

10.6 Feature Importance

Feature importance is not applicable here. Neither the PLA nor scikit-learn's MLP provides a built-in importance score, and the inputs are raw pixels, so no individual pixel has a meaning of its own.

11 Discussion

11.1 PLA vs MLP

Aspect	PLA	MLP (tuned)
Boundary	Linear only	Non-linear
Test accuracy / F1	23.2% / 0.209	52.9% / 0.531
Convergence	Error keeps fluctuating around 50%	Loss reaches almost 0 in about 30 epochs
Strengths	Simple, fast, easy to explain	Learns complex shapes, gives probabilities
Weaknesses	Fails on non-separable classes, no probabilities	Needs tuning, more computation, overfits

Tuning also helped: the untuned MLP reached 42.4% test accuracy and the tuned one 52.9%, an increase of about 10 points, with much faster convergence.

11.2 Observation Questions

Why does PLA underperform compared to MLP? Characters from 62 classes are not linearly separable in pixel space, and the PLA can only place one straight boundary per class. Its training error never goes to zero (training accuracy 48.9%) because later updates undo earlier ones. The hidden layers of the MLP combine pixels into strokes and curves, which allows curved boundaries.

Which hyperparameters had the most impact? The learning rate (0.151) and the activation function (0.135) had the largest effect, followed by the number of hidden layers (0.092). The optimizer (0.043) and batch size (0.036) changed the result much less.

Did optimizer choice (SGD vs Adam) affect convergence? Adam was better on average (0.301 against 0.259) since it works over a wider range of learning rates, while SGD was too slow at 0.001. With a well-chosen rate of 0.1, SGD reached almost the same accuracy (0.509 against 0.518) and converged as quickly. The optimizer therefore matters mainly through its sensitivity to the learning rate.

Did more hidden layers always improve results? No. The mean validation accuracy dropped from 0.329 to 0.273 to 0.238 for one, two and three layers, and the best network of each depth scored almost the same (0.509, 0.518, 0.502). With only about 44 training images per class, deeper networks have more weights to fit, are harder to train in 50 epochs and overfit more.

Did the MLP overfit? How can it be reduced? Yes. The tuned MLP has 99.9% training accuracy but 52.9% test accuracy, and its training loss goes to zero while validation accuracy stays flat. Overfitting can be reduced with L2 regularization (larger α), early stopping, dropout, a smaller network, and above all more data or data augmentation such as small shifts, rotations and stroke-width changes.

11.3 Limitations and Improvements

The dataset is small (55 images per class), so the gap between training and test accuracy is large. The 20×20 raw pixels are also a weak representation, since the characters occupy only the centre of the image. The validation set has 546 images, so differences of one or two percentage points between configurations are within noise, and only a single split was used. The MSE comparison used a regressor with a linear output layer, so it is not an exact swap of the loss alone. Possible improvements are data augmentation, cross-validation, regularization with early stopping, better features such as HOG, and a convolutional network, which suits image data better than a fully connected one.

12 Conclusion

A one-vs-rest PLA and a tuned MLP were compared on 3,410 handwritten character images from 62 classes. The PLA reached 23.2% test accuracy (macro F1 0.209, ROC-AUC 0.825) and its training error never converged, which shows that the classes are not linearly separable. The best MLP (ReLU, Adam, learning rate 0.01, batch size 128, hidden layers 128 and 64, cross-entropy loss) reached 52.9% accuracy (macro F1 0.531, ROC-AUC 0.954), and tuning gave about 10 points over the default MLP. Learning rate and activation function mattered most, while extra hidden layers gave no gain. The large gap between training accuracy (99.9%) and test accuracy shows that overfitting is the main remaining problem, which more data or augmentation should address.

13 References

References

- [1] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
- [2] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [3] D. Dave, “English handwritten characters dataset,” Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/dhruvildave/english-handwritten-characters-dataset>